

Programmes numériques utilisés

Les casques

- Méthode de Monte-Carlo (p°)

Les pneus

- Importation des données dans Python (p°)
- Dérivation discrète (p°)
- Interpolation polynomiale (p°)
- Dichotomie (p°)

- Création de balises

Fichier GPX avant et après modification :

```
k?xml version="1.0" encoding="UTF-8">
<gpx xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.topografix.com/GPX/1/1 http://www.topografix.com/GPX/1/1/gpx.xsd
http://www.garmin.com/xmlschemas/GpxExtensions/v3
http://www.garmin.com/xmlschemas/TrackPointExtension/v1
http://www.garmin.com/xmlschemas/TrackPointExtension/v1.xsd" creator="StravaGPX" version="1.1"
xmlns="http://www.topografix.com/GPX/1/1"
xmlns:gpxtpx="http://www.garmin.com/xmlschemas/TrackPointExtension/v1"
xmlns:gpox="http://www.garmin.com/xmlschemas/GpxExtensions/v3">
  <metadata>
    <time>2024-03-03T20:26:49Z</time>
  </metadata>
  <trk>
    <name>Vélo de nuit</name>
    <type>cycling</type>
    <trkseg>
      <trkpt lat="48.8171000" lon="2.3147160">
        <ele>75.3</ele>
        <time>2024-03-03T20:26:49Z</time>
        <extensions>
          <gpxtpx:TrackPointExtension>
            <gpxtpx:hr>88</gpxtpx:hr>
          </gpxtpx:TrackPointExtension>
        </extensions>
      </trkpt>
      <trkpt lat="48.8170990" lon="2.3147250">
        <ele>75.3</ele>
        <time>2024-03-03T20:26:52Z</time>
        <extensions>
          <gpxtpx:TrackPointExtension>
            <gpxtpx:hr>87</gpxtpx:hr>
          </gpxtpx:TrackPointExtension>
        </extensions>
      </trkpt>
      <trkpt lat="48.8171030" lon="2.3147340">
        <ele>75.3</ele>
        <time>2024-03-03T20:26:53Z</time>
        <extensions>
          <gpxtpx:TrackPointExtension>
            <gpxtpx:hr>88</gpxtpx:hr>
          </gpxtpx:TrackPointExtension>
        </extensions>
      </trkpt>
      <trkpt lat="48.8171050" lon="2.3147480">
        <ele>75.3</ele>
        <time>2024-03-03T20:26:54Z</time>
      </trkpt>
    </trkseg>
  </trk>
</gpx>
```

```
>?xml version="1.0" encoding="UTF-8">
<gpx xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.topografix.com/GPX/1/1 http://www.topografix.com/GPX/1/1/gpx.xsd
http://www.garmin.com/xmlschemas/GpxExtensions/v3
http://www.garmin.com/xmlschemas/TrackPointExtension/v1
http://www.garmin.com/xmlschemas/TrackPointExtension/v1.xsd" creator="StravaGPX" version="1.1"
xmlns="http://www.topografix.com/GPX/1/1"
xmlns:gpxtpx="http://www.garmin.com/xmlschemas/TrackPointExtension/v1"
xmlns:gpox="http://www.garmin.com/xmlschemas/GpxExtensions/v3">
  <metadata>
    <time>2024-03-03T20:26:49>>/time>
  </metadata>
  <trk>
    <name>Vélo de nuit</name>
    <type>cycling</type>
    <trkseg>
      <trkpt lat="48.8171000" lon="2.3147160">
        <ele>75.3>/ele>
        <time>2024-03-03T20:26:49>>/time>
        <extensions>
          <gpxtpx:TrackPointExtension>
            <gpxtpx:hr>88>/gpxtpx:hr>
          </gpxtpx:TrackPointExtension>
        </extensions>
      </trkpt>
      <trkpt lat="48.8170990" lon="2.3147250">
        <ele>75.3>/ele>
        <time>2024-03-03T20:26:52>>/time>
        <extensions>
          <gpxtpx:TrackPointExtension>
            <gpxtpx:hr>87>/gpxtpx:hr>
          </gpxtpx:TrackPointExtension>
        </extensions>
      </trkpt>
      <trkpt lat="48.8171030" lon="2.3147340">
        <ele>75.3>/ele>
        <time>2024-03-03T20:26:53>>/time>
        <extensions>
          <gpxtpx:TrackPointExtension>
            <gpxtpx:hr>88>/gpxtpx:hr>
          </gpxtpx:TrackPointExtension>
        </extensions>
      </trkpt>
      <trkpt lat="48.8171050" lon="2.3147480">
        <ele>75.3>/ele>
        <time>2024-03-03T20:26:54>>/time>
      </trkpt>
    </trkseg>
  </trk>
</gpx>
```

- Programme utilisé :

```
1 import os
2 import shutil
3
4 def copie(entree,sortie): #Effectue la copie du fichier original
5     shutil.copy(entree,sortie)
6
7 def remplacer(chemin,ancien_car,nouveau_car): #Remplace les caractères
8     with open(chemin,'r') as f:
9         texte=f.read()
10
11     texte=texte.replace(ancien_car,nouveau_car)
12
13     with open(chemin,'w') as f:
14         f.write(texte)
15
16 #Si le chemin n'existe pas, il est créé
17 sortie="D:\\Ecole\\prépa\\psi\\Mesures_pneus_pression_tipe\\mesures_3\\20psiconv"
18 if not os.path.exists(sortie):
19     os.makedirs(sortie)
20
21 for i in range(1,4): #Effectue l'opération pour les fichiers nommés 1,2 et 3
22     entree=f"D:\\Ecole\\prépa\\psi\\Mesures_pneus_pression_tipe\\mesures_3\\20psi\\{i}.gpx"
23     sortie=f"D:\\Ecole\\prépa\\psi\\Mesures_pneus_pression_tipe\\mesures_3\\20psiconv\\{i}.gpx"
24
25     #Copie du contenu en entrée vers la sortie
26     copie(entree,sortie)
27
28     #Remplacement des caractères dans le fichier de sortie
29     remplacer(sortie,'"', '>'')
30     remplacer(sortie,'<', '>')
31     remplacer(sortie,'Z', '>')
```

- Dérivation discrète

```

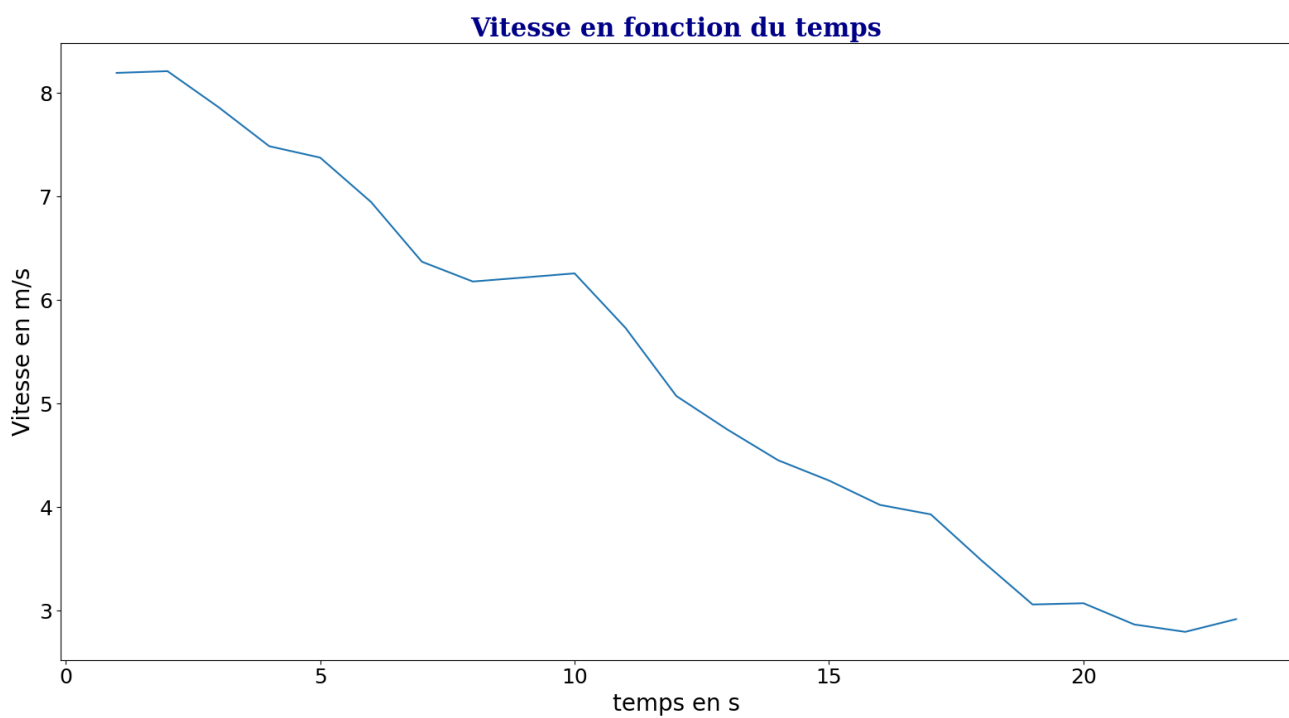
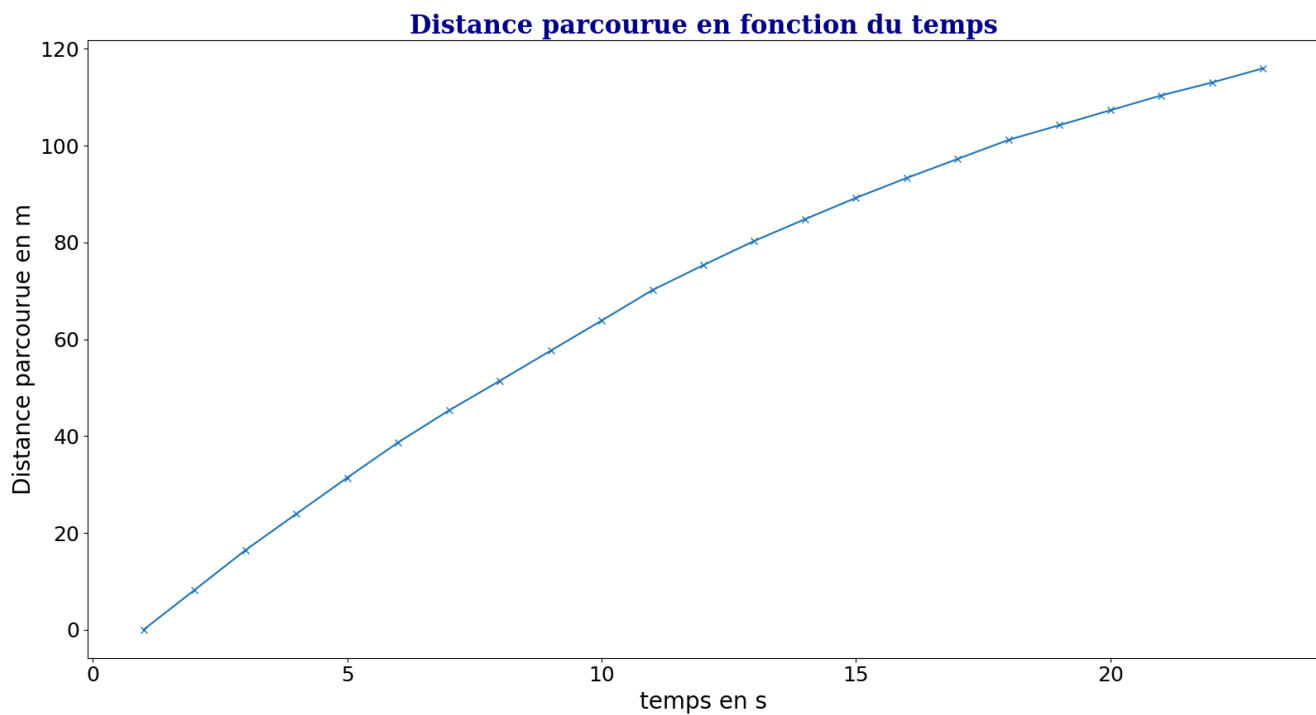
1  ##Données
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import scipy.integrate as sp
5  from scipy.integrate import odeint
6
7  #Données
8  M=84 #masse du vélo+cycliste
9  g=9.81 #accélération de la pesanteur
10 rho=1.212 #masse volumique de l'air
11 S=0.5644 #surface frontale
12 theta=0*np.pi/180 #angle de la pente
13
14 #Mesures
15
16 DTOT40psil=[8.17771470773408, 17.637142632121083, 28.02410197888254, 38.97656867823143, 49.363046641202715,
59.00232569537784, 68.14469865424132, 77.0471195408489, 85.87645731842237, 94.56805059317196,
103.19911011877284, 110.65565774538418, 117.80421317152441, 124.84822645132465]
17
18 TEMPS_TOT40psil=[10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0, 17.0, 18.0, 19.0, 20.0, 21.0, 22.0, 23.0]
19
20 DTOT40psi2=[43.02146375768431, 54.754420383768775, 66.38108808658114, 77.04295652974012, 86.82053947501377,
96.8778927826605, 106.0922079530244, 114.35737674796712, 121.95356272786715, 127.83935366516205]
21
22 TEMPS_TOT40psi2=[5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0]
23
24 DTOT40psi3=[85.01504041180081, 92.68347132322445, 100.56563330716587, 107.32021983163071,
113.84671768206046, 120.18660148173849, 126.34263287093934, 132.04164807451212, 137.57129808585276,
141.3747551414146]
25
26 TEMPS_TOT40psi3=[35.0, 36.0, 37.0, 38.0, 39.0, 40.0, 41.0, 42.0, 43.0, 44.0]
27
28 DTOT40psi4=[60.16383557363731, 71.9451678870685, 83.24217730199341, 93.83609888404895, 103.19024859730455,
112.89548392337255, 121.69382524416669, 130.20501450547061, 138.11863666958678, 145.6813810937239,
150.7633842220559]
29
30 TEMPS_TOT40psi4=[6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0]
31
32 DTOT40psi5=[0, 8.193048146487737, 16.41983787224445, 23.9195320582378, 31.390319215738646,
38.66982643565665, 45.28383115523552, 51.40946807874154, 57.64135485663523, 63.84440626351948,
70.15594811503108, 75.30705361762745, 80.30217639171786, 84.80797900520017, 89.20800201600761,
93.32222011373835, 97.25198715025826, 101.18302632050377, 104.22009770488258, 107.30363180949112,
110.3646464143657, 113.03795753683696, 115.95669610641956]
33
34 TEMPS_TOT40psi5=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0,
17.0, 18.0, 19.0, 20.0, 21.0, 22.0, 23.0]
35
36
37
38
39 DTOT20psil=[0, 7.913622163538733, 15.58205307504593, 22.517104920463442, 29.344519280860048,
35.65606113222654, 41.35419892333825, 46.61338550003546, 51.442132068399275, 56.05505311418548,
60.28923575507805, 62.79791119057468]
40
41 TEMPS_TOT20psil=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0]
42
43 DTOT20psi2=[0, 9.565362783015487, 18.0765520443194, 26.501735371024015, 34.28618544163287,
41.640047547923416, 48.74436053379993, 55.487779741298596, 62.31592640434493, 68.18403558909975,
71.50281074190744]
44
45 TEMPS_TOT20psi2=[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0]
46
47 DTOT20psi3=[73.17662324995837, 77.57550975661039, 81.48763153372684, 85.46120305871266, 88.91004365688137,
91.89539616300742, 94.39419488546137, 96.9805265610597, 99.23205540222602, 100.85108025981145]
48
49 TEMPS_TOT20psi3=[27.0, 28.0, 29.0, 30.0, 31.0, 32.0, 33.0, 34.0, 35.0, 36.0]
50
51 ##
52 def filtrer_donnees_moyennage(liste, taille_fenetre=3):
53     """
54     Filtre les données en utilisant un filtre de moyennage.
55
56     Paramètres:

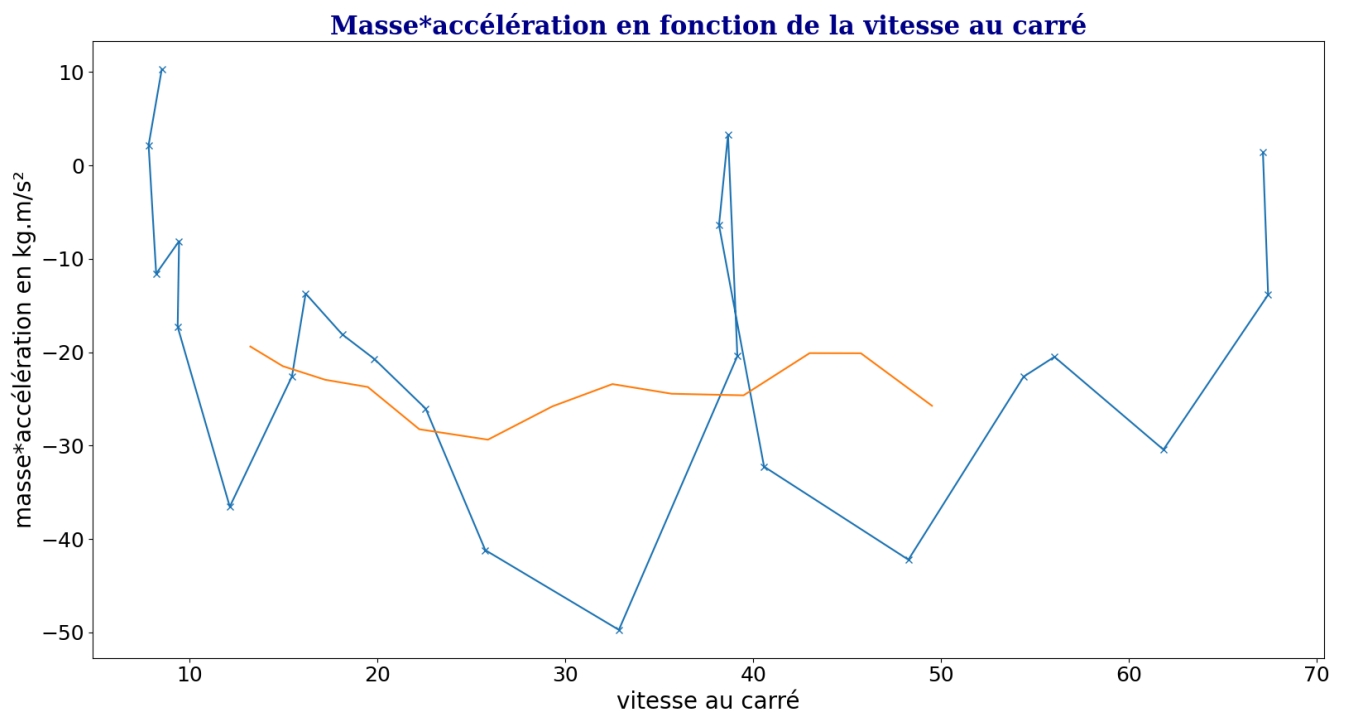
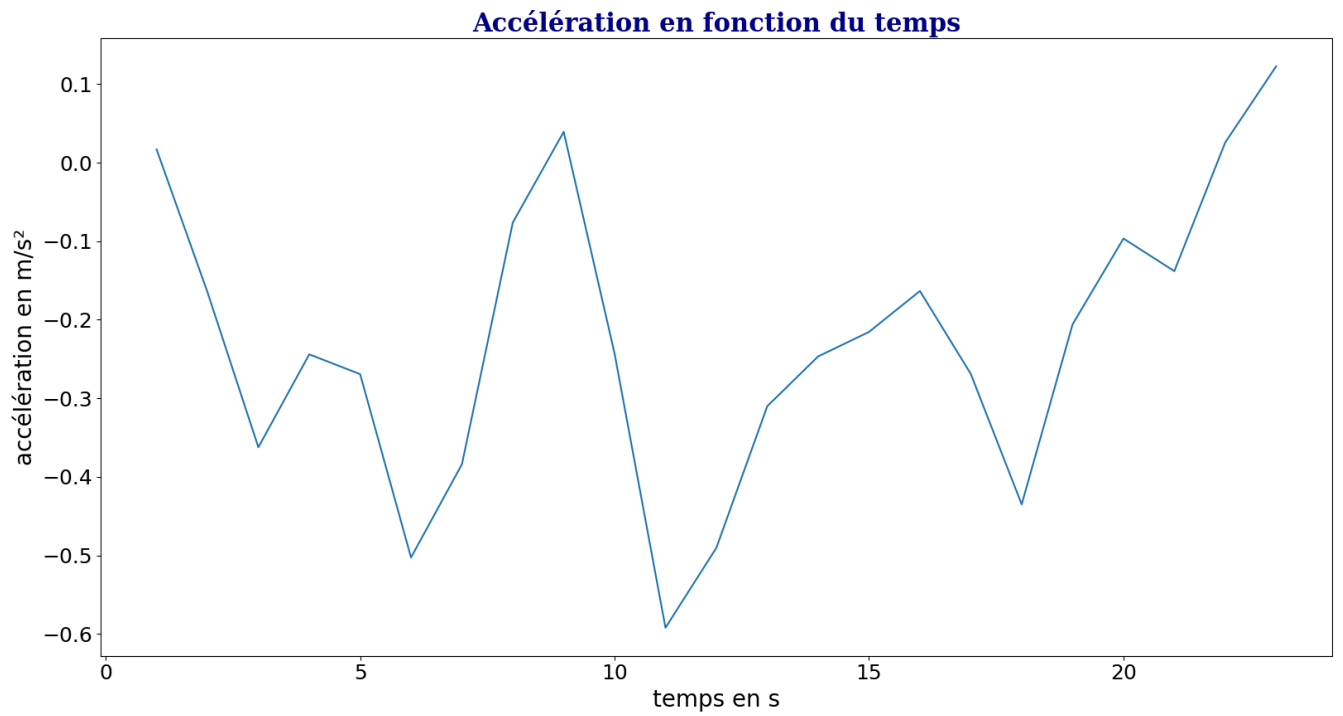
```

```

57 liste (list): Une liste de valeurs numériques.
58 taille_fenetre (int): La taille de la fenêtre de moyennage.
59
60 Retourne:
61 list: Une nouvelle liste représentant les valeurs lissées.
62 ""
63 fenetre = np.ones(taille_fenetre) / taille_fenetre
64 liste_lissee = np.convolve(liste, fenetre, mode='valid')
65
66 # Ajouter les points de bord pour garder la même longueur que la liste originale
67 pad_size = (taille_fenetre - 1) // 2
68 liste_lissee = np.pad(liste_lissee, (pad_size, pad_size), mode='edge')
69
70 return liste_lissee.tolist()
71 ##
72 def derivee_discrete(liste):
73     n = len(liste)
74     if n < 3:
75         raise ValueError("La liste doit contenir au moins trois éléments pour utiliser les différences
centrées.")
76     derivee = [(liste[i + 1] - liste[i - 1]) / 2 for i in range(1, n - 1)]
77     derivee = [(liste[1] - liste[0])] + derivee + [(liste[-1] - liste[-2])]
78     return derivee
79
80 Dfiltre=filtrer_donnees_moyennage(DTOT40psi5,6)
81 V=derivee_discrete(DTOT40psi5)
82 VL=derivee_discrete(Dfiltre)
83 ACC=derivee_discrete(V)
84 ACCL=derivee_discrete(VL)
85
86 MACCL=[ACCL[i]*M for i in range(0,len(ACCL))]
87 VQL=[VL[i]**2 for i in range(0,len(VL))]
88 MACC=[ACC[i]*M for i in range(0,len(ACC))]
89 VQ=[V[i]**2 for i in range(0,len(V))]
90 ##
91 plt.plot(TEMPS_TOT40psi5,DTOT40psi5,marker='x')
92 plt.xlabel("temps en s",size=20)
93 plt.ylabel("Distance parcourue en m", size=20)
94
95 plt.title("Distance parcourue en fonction du temps",fontdict={"family": "serif", "color": "darkblue",
"weight": "bold", "size": 22})
96 plt.xticks(fontsize=18)
97 plt.yticks(fontsize=18)
98 plt.show()
99 ##
100 plt.plot(TEMPS_TOT40psi5,V)
101 plt.title("Vitesse en fonction du temps",fontdict={"family": "serif", "color": "darkblue", "weight":
"bold", "size": 22},)
102 plt.xlabel("temps en s",size=20,)
103 plt.ylabel("Vitesse en m/s", size=20)
104 plt.xticks(fontsize=18)
105 plt.yticks(fontsize=18)
106 plt.show()
107 ##
108 plt.plot(TEMPS_TOT40psi5,ACC)
109 plt.xlabel("temps en s",size=20)
110 plt.ylabel("accélération en m/s²",size=20)
111 plt.title("Accélération en fonction du temps",fontdict={"family": "serif", "color": "darkblue", "weight":
"bold", "size": 22})
112 plt.xticks(fontsize=18)
113 plt.yticks(fontsize=18)
114 plt.show()
115 ##
116 plt.plot(VQ,MACC, marker='x')
117 del VQL[-5:]
118 del MACCL[-5:]
119 del VQL[:4]
120 del MACCL[:4]
121
122 P=np.polyfit(VQ,MACC,1)
123 Cr=abs(P[1]/(M*g))
124 print("Cr = ",round(Cr,4))
125 P=np.polyfit(VQL,MACCL,1)
126 Cr=abs(P[1]/(M*g))
127 Cx=abs(2*P[0]/(rho*S))
128 print("Cr = ",round(Cr,4)," (Après lissage)")
129 print("Cx = ",round(Cx,4)," (Après lissage)")
130
131 plt.plot(VQL,MACCL)
132 plt.xlabel("vitesse au carré",size=20)
133 plt.ylabel("masse*accélération en kg.m/s²",size=20)
134 plt.title("Masse*accélération en fonction de la vitesse au carré",fontdict={"family": "serif", "color":
"darkblue", "weight": "bold", "size": 22})
135 plt.xticks(fontsize=18)
136 plt.yticks(fontsize=18)
137 plt.show()

```





```
>>> (executing cell "" (line 116 of "derivation discrete.py"))
Cr = 0.0189
Cr = 0.0288 (Après lissage)
```

- Conversion en listes distance et temps des coordonnées GPS

```

1 import matplotlib.pyplot as plt
2 import scipy.special as special
3 import numpy as np
4 txt_file_path = r"D:\Ecole\prépa\psi\Mesures_pneus_pression_tipe\2_bars.txt"
5
6 #"C:\Users\nbras\OneDrive\Bureau\donnees_velo_sauvegarde.txt"
7 ##
8
9 #données numériques
10
11 g=9.81 #accélération de la pesanteur en m/s-2
12 rho=1.225 #masse volumique de l'air en kg/m3
13 Aire=0.5 #aire projetée totale en m²
14 m=65+15.5 #masse totale en kg
15
16
17 mode_path = r"D:\Ecole\prépa\psi\Mesures_pneus_pression_tipe\2_bars.txt"
18 f = open(txt_file_path,'r')
19
20 LAT=[]
21 LON=[]
22 ELE=[]
23 TEMPS=[]
24 DT=[]
25 TEMPS_TOT=[0]
26 lat_balise="lat=" #il faut remplacer dans le fichier les " par des _
27 ele_balise=">ele>" #il faut remplacer dans le fichier les < par des >
28 temps_balise=">time>"
29 balise1="_"
30 balise2=">" #il faut remplacer dans le fichier les Z par des >
31
32 #liste latitude et longitude
33 with open(txt_file_path) as f:
34     for line in f:
35         pos_begin=line.find(lat_balise)
36         if pos_begin!=-1:
37             A=line.split(balise1)
38             LAT.append(float(A[1]))
39             LON.append(float(A[3]))
40 #liste élévation
41 with open(txt_file_path) as f:
42     for line in f:
43         pos_begin=line.find(ele_balise)
44         if pos_begin!=-1:
45             A=line.split(balise2)
46             ELE.append(float(A[2]))
47 #liste temps
48 with open(txt_file_path) as f:
49     for line in f:
50         pos_begin=line.find(temps_balise)
51         if pos_begin!=-1:
52             A=line.split(balise2)
53             TEMPS.append(A[2])
54
55
56 del LAT[:40]
57 del LON[:40]
58 del ELE[:40]
59 del TEMPS[:40]#le premier temps apparait 2 fois dans le fichier
60
61 del LAT[-4:]
62 del LON[-4:]
63 del ELE[-4:]

```

```

64 del TEMPS[-4:]
65
66 R=6371.009*10**3 #rayon de la Terre
67 LAT=[i*R*np.pi/180 for i in LAT]#conversion des degrés en mètre
68 LON=[i*R*np.pi/180 for i in LON]
69 #l'élévation est déjà en mètres
70
71 #liste delta-temps et temps accumulé
72 for j in range(0,len(TEMPS)-1):
73     if (float(TEMPS[j+1][17:19])<float(TEMPS[j][17:19])):
74         DT.append(abs(float(TEMPS[j+1][17:19])-float(TEMPS[j][17:19])+60))
75     else:
76         DT.append(abs(float(TEMPS[j+1][17:19])-float(TEMPS[j][17:19])))
77     TEMPS_TOT.append(TEMPS_TOT[j]+DT[j])
78 TEMPS_TOT.pop(0)
79
80 #calcul delta-distance
81 D=[]
82 DTOT=[0]
83
84 for i in range(0,len(LAT)-1):
85     d=np.sqrt((LAT[i+1]-LAT[i])**2+(LON[i+1]-LON[i])**2+(ELE[i+1]-ELE[i])**2)
86     D.append(d)
87     DTOT.append(DTOT[i]+D[i])
88

```

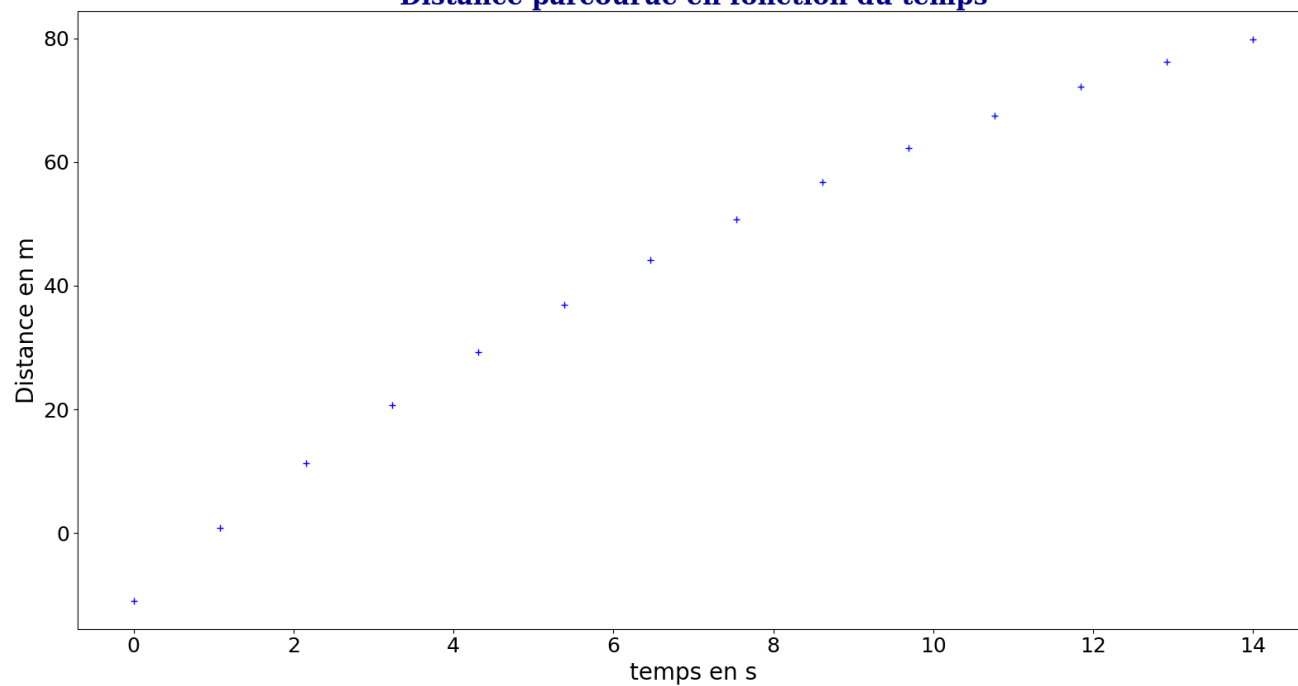

- Interpolation polynomiale

```

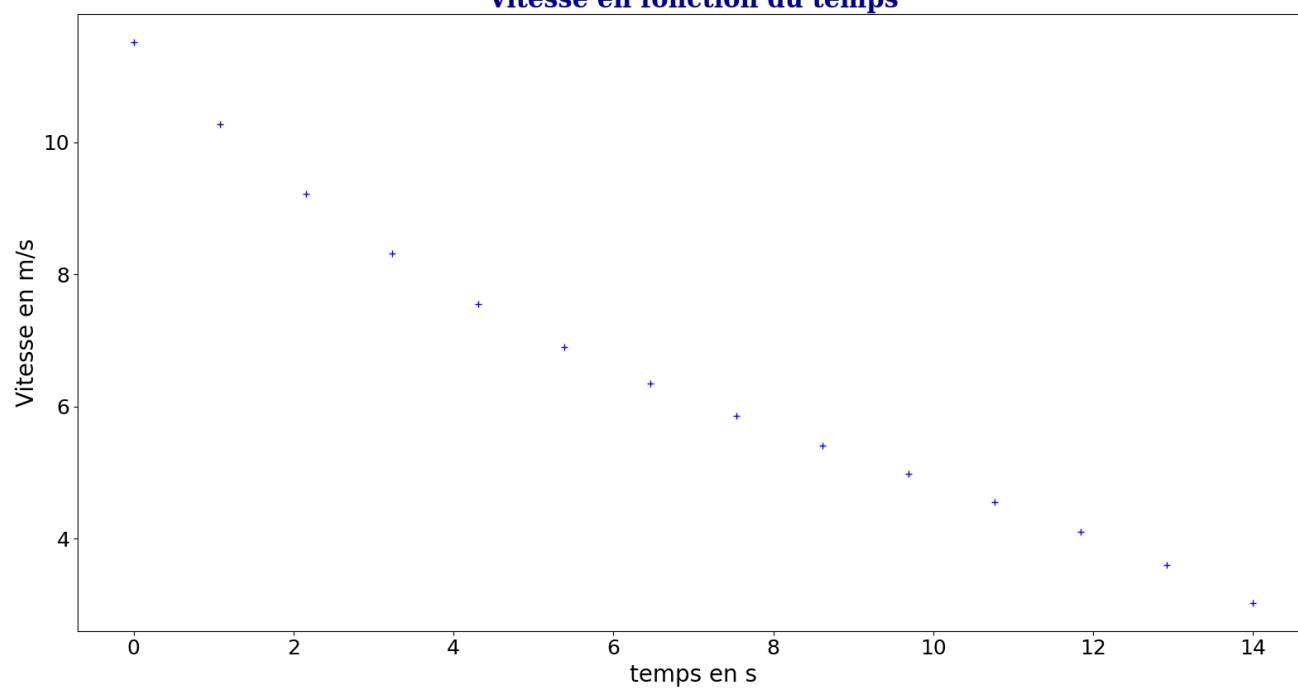
89 #régression linéaire
90
91 L=np.polyfit(TEMPS_TOT,DTOT,4)
92 t=np.linspace(0,TEMPS_TOT[-1],len(TEMPS_TOT))
93
94
95 def position(t):
96     return L[0]*t**4+L[1]*t**3+L[2]*t**2+L[3]*t+L[4]
97
98 plt.plot(t,position(t),'b+')
99 plt.title("Distance parcourue en fonction du temps",fontdict={"family": "serif", "color": "darkblue",
100 "weight": "bold", "size": 22})
101 plt.xlabel("temps en s",size=20,)
102 plt.ylabel("Distance en m", size=20)
103 plt.xticks(fontsize=18)
104 plt.yticks(fontsize=18)
105 plt.show()
106
107 def vitesse(t):
108     return (4*L[0]*t**3+3*L[1]*t**2+2*L[2]*t+L[3])
109
110 plt.plot(t,vitesse(t),'b+')
111 plt.title("Vitesse en fonction du temps",fontdict={"family": "serif", "color": "darkblue", "weight":
112 "bold", "size": 22},)
113 plt.xlabel("temps en s",size=20,)
114 plt.ylabel("Vitesse en m/s", size=20)
115 plt.xticks(fontsize=18)
116 plt.yticks(fontsize=18)
117 plt.show()
118
119 def acceleration(t):
120     return (12*L[0]*t**2+6*L[1]*t+2*L[2])
121
122 plt.plot(t,acceleration(t),'b+')
123 plt.xlabel("temps en s",size=20)
124 plt.ylabel("accélération en m/s²",size=20)
125 plt.title("Accélération en fonction du temps",fontdict={"family": "serif", "color": "darkblue", "weight":
126 "bold", "size": 22})
127 plt.xticks(fontsize=18)
128 plt.yticks(fontsize=18)
129 plt.show()
130
131 VQ=[]
132 ACC=[]
133
134 for i in range(0,len(t)):
135     VQ.append(vitesse(t[i])**2)
136     ACC.append(acceleration(t[i]))
137
138 #On supprime les 6 dernières valeurs qui sont aberrantes
139 del VQ[-6:]
140 del ACC[-6:]
141
142 VQ=np.array(VQ)
143 ACC=np.array(ACC)
144
145 P=np.polyfit(VQ,ACC,1)
146 print("Cr = ",round(abs(P[1]/g),4))
147
148 plt.plot(VQ,ACC,'b+')
149 plt.plot(VQ,P[0]*VQ+P[1])
150 plt.xlabel("vitesse au carré",size=20)
151 plt.ylabel("accélération en m/s²",size=20)
152 plt.title("accélération en fonction de la vitesse au carré",fontdict={"family": "serif", "color":
153 "darkblue", "weight": "bold", "size": 22})
154 plt.xticks(fontsize=18)
155 plt.yticks(fontsize=18)
156 plt.show()
157 plt.show()
158 plt.close()

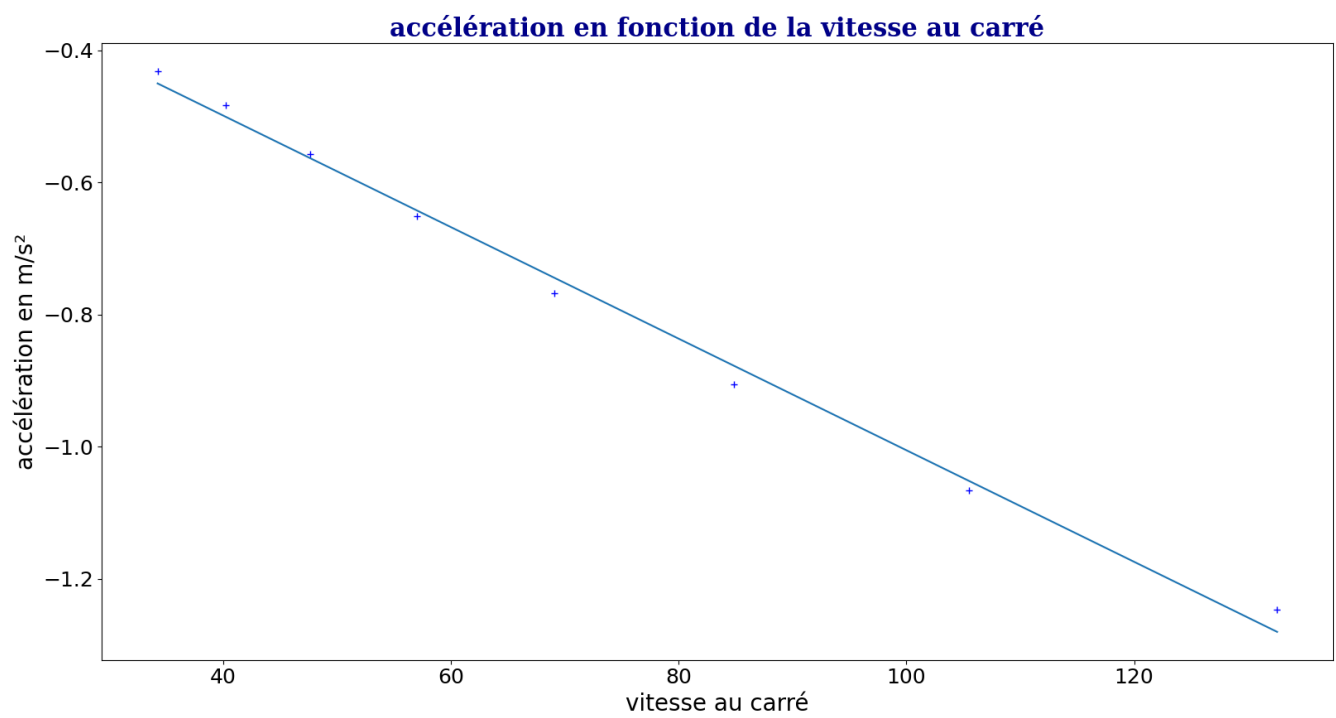
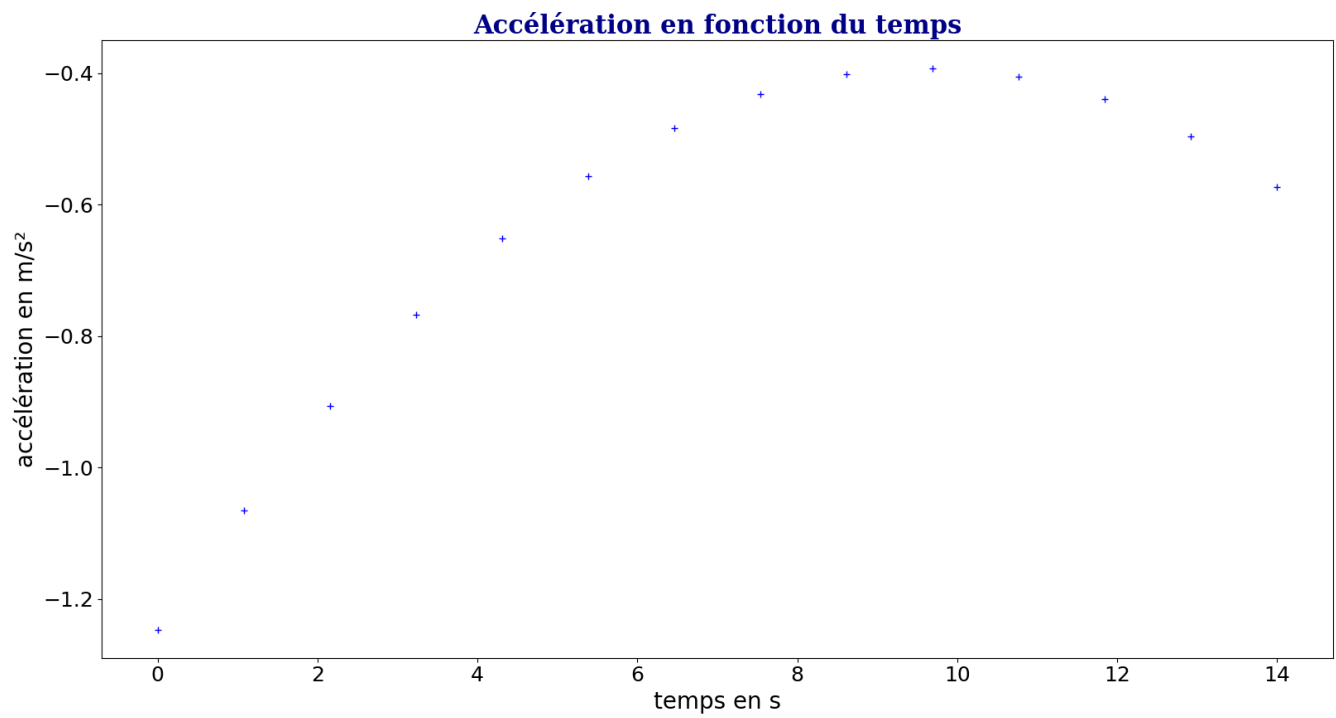
```

Distance parcourue en fonction du temps



Vitesse en fonction du temps





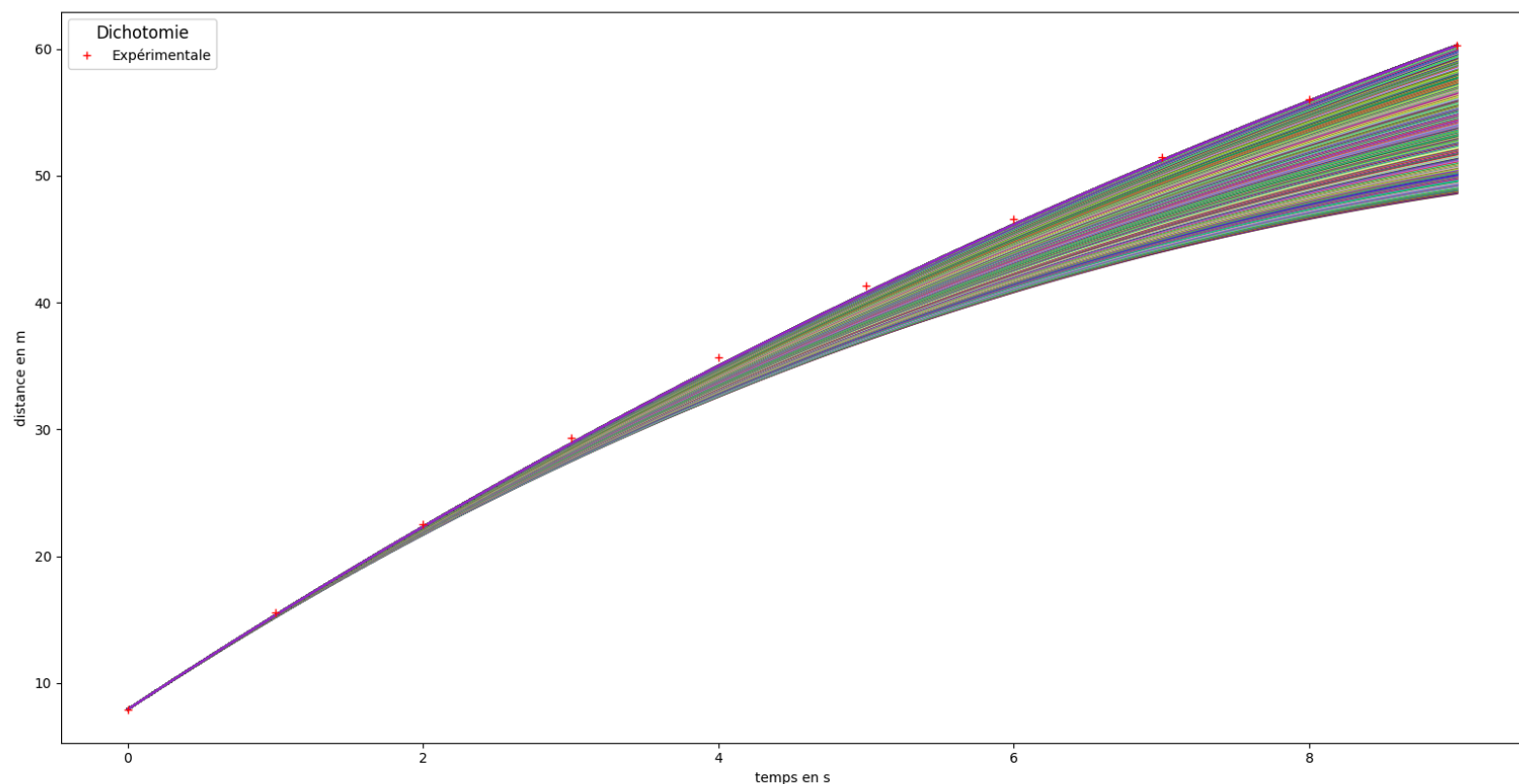
```
>>> (executing cell "" (line 8 of "2_bars_poly.py"))  
Cr = 0.0164
```

- Méthode de dichotomie

```

958 ##Fonction 20 psi 1
959
960 DTOT20psil=[7.913622163538733, 15.58205307504593, 22.517104920463442, 29.344519280860048, 35.65606113222654, 41.35419892333825,
46.61338550003546, 51.442132068399275, 56.05505311418548, 60.28923575507805]
961
962 TEMPS_TOT20psil=[2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0]
963
964 t=np.linspace(TI,TF,N+1)
965
966 for i in range(0,len(TEMPS_TOT20psil)):
967     TEMPS_TOT20psil[i]=TEMPS_TOT20psil[i]-2.0
968
969 TI=TEMPS_TOT20psil[0]
970 TF=TEMPS_TOT20psil[-1]
971 N=100
972 T=(TF-TI)/N
973 x0=DTOT20psil[0]
974 v0=(DTOT20psil[1]-DTOT20psil[0])/(TEMPS_TOT20psil[1]-TEMPS_TOT20psil[0])
975
976 Cx=1.1966
977
978 plt.close()
979 def dichotomie(P, app, N, x0, v0):
980     m=0
981     Cr=0.04
982     while m<P:
983         def F(X,t):
984             dXdt=X[1]
985             dVdt=-((1/2)*Cx*rho*(X[1]**2)*S)/M-g*(Cr+np.sin(theta))
986             return [dXdt,dVdt]
987         ODEINT=odeint(F,np.array([x0,v0]),t)
988         X1=[]
989         for i in range(0,len(ODEINT)):
990             X1.append(ODEINT[i][0])
991         if DTOT20psil[-1]<X1[-1]:
992             Cr+=app
993         else:
994             Cr-=app
995         print(m,"Cr = ",round(Cr,6))
996
997         plt.plot(t,X1,color=np.random.rand(3,))
998         m+=1
999         print('erreur en mètres (par rapport à la dernière position) : ',abs(round(DTOT20psil[-1]-X1[-1],3)))
1000     return round(Cr,6)
1001
1002 dichotomie(500,1/10000,N,x0,v0)
1003
1004 plt.plot(TEMPS_TOT20psil,DTOT20psil,'r+',label='Expérimentale')
1005 plt.xlabel("temps en s")
1006 plt.ylabel("distance en m")
1007 plt.legend(loc='upper left', title='Dichotomie', title_fontsize='large')
1008 plt.show()

```



```

>>> (executing cell "Fonction 20 psi 1" (line 959 of "TIPE simula
0 Cr = 0.0399
erreur en mètres (par rapport à la dernière position) : 11.689
1 Cr = 0.0398
erreur en mètres (par rapport à la dernière position) : 11.654
2 Cr = 0.0397
erreur en mètres (par rapport à la dernière position) : 11.618
3 Cr = 0.0396
erreur en mètres (par rapport à la dernière position) : 11.582
4 Cr = 0.0395
erreur en mètres (par rapport à la dernière position) : 11.547
5 Cr = 0.0394
erreur en mètres (par rapport à la dernière position) : 11.511
6 Cr = 0.0393
erreur en mètres (par rapport à la dernière position) : 11.476
7 Cr = 0.0392
erreur en mètres (par rapport à la dernière position) : 11.44
8 Cr = 0.0391
erreur en mètres (par rapport à la dernière position) : 11.405
9 Cr = 0.039
erreur en mètres (par rapport à la dernière position) : 11.369
10 Cr = 0.0389
erreur en mètres (par rapport à la dernière position) : 11.334
11 Cr = 0.0388
erreur en mètres (par rapport à la dernière position) : 11.298
12 Cr = 0.0387
erreur en mètres (par rapport à la dernière position) : 11.263
13 Cr = 0.0386
erreur en mètres (par rapport à la dernière position) : 11.227
14 Cr = 0.0385
erreur en mètres (par rapport à la dernière position) : 11.192
15 Cr = 0.0384
erreur en mètres (par rapport à la dernière position) : 11.156
16 Cr = 0.0383
erreur en mètres (par rapport à la dernière position) : 11.121
17 Cr = 0.0382
erreur en mètres (par rapport à la dernière position) : 11.085
18 Cr = 0.0381
erreur en mètres (par rapport à la dernière position) : 11.05
19 Cr = 0.038
erreur en mètres (par rapport à la dernière position) : 11.014
20 Cr = 0.0379
erreur en mètres (par rapport à la dernière position) : 10.979
21 Cr = 0.0378
495 Cr = 0.0066
erreur en mètres (par rapport à la dernière position) : 0.033
496 Cr = 0.0067
erreur en mètres (par rapport à la dernière position) : 0.001
497 Cr = 0.0066
erreur en mètres (par rapport à la dernière position) : 0.033
498 Cr = 0.0067
erreur en mètres (par rapport à la dernière position) : 0.001
499 Cr = 0.0066
erreur en mètres (par rapport à la dernière position) : 0.033

```

- Simulation de Monte-Carlo pour les casques

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import numpy.random as rd
4
5 N=1000000
6 rho=1.293
7
8 S1=1.392*10**-2
9 S2=1.298*10**-2
10 S3=1.142*10**-2
11
12
13 F10=0.155
14 F20=0.135
15 F30=0.12
16
17 dF=0.005
18
19 V0=7.71
20 dV=0.43
21
22 F1=F10+rd.uniform(-dF,dF,N)
23 F2=F20+rd.uniform(-dF,dF,N)
24 F3=F30+rd.uniform(-dF,dF,N)
25
26 V=V0+rd.uniform(-dV,dV,N)
27
28 Cx1=2*F1/(rho*S1*V**2)
29 Cx2=2*F2/(rho*S2*V**2)
30 Cx3=2*F3/(rho*S3*V**2)
31
32 Cx1moy= np.mean(Cx1)
33 uCx1=np.std(Cx1,ddof=1)
34 Cx2moy= np.mean(Cx2)
35 uCx2=np.std(Cx2,ddof=1)
36 Cx3moy= np.mean(Cx3)
37 uCx3=np.std(Cx3,ddof=1)
38
39 print("Casque sphérique : Cx = ",round(Cx1moy,3),"±",round(uCx1,3))
40 print("Casque intermédiaire : Cx = ",round(Cx2moy,3),"±",round(uCx2,3))
41 print("Casque à pointe longue : Cx = ",round(Cx3moy,3),"±",round(uCx3,3))
42
43 plt.figure(1)
44 plt.title("Coefficient de traînée : algorithme de Monte Carlo")
45
46 plt.hist(Cx1, bins='rice', color='red', density=True, histtype='stepfilled', label="Casque sphérique")
47
48 plt.hist(Cx2, bins='rice', color='blue', density=True, histtype='step', label="Casque intermédiaire")
49 plt.hist(Cx3, bins='rice', color='green', density=True, histtype='step', label="Casque à pointe longue")
50 plt.xlabel("Cx")
51 plt.ylabel("densité de probabilité")
52 plt.legend()
53 plt.show()
```

>>> (executing cell "" (line 1 of "monte carlo pour les casques.py"))

Casque sphérique : Cx = 0.291 ± 0.02

Casque intermédiaire : Cx = 0.271 ± 0.018

Casque à pointe longue : Cx = 0.274 ± 0.019

Coefficient de traînée : algorithme de Monte Carlo

